In [24]:

```
# This cell is added by sphinx-gallery
# It can be customized to whatever you like
%matplotlib inline
%pip install tensorflow
```

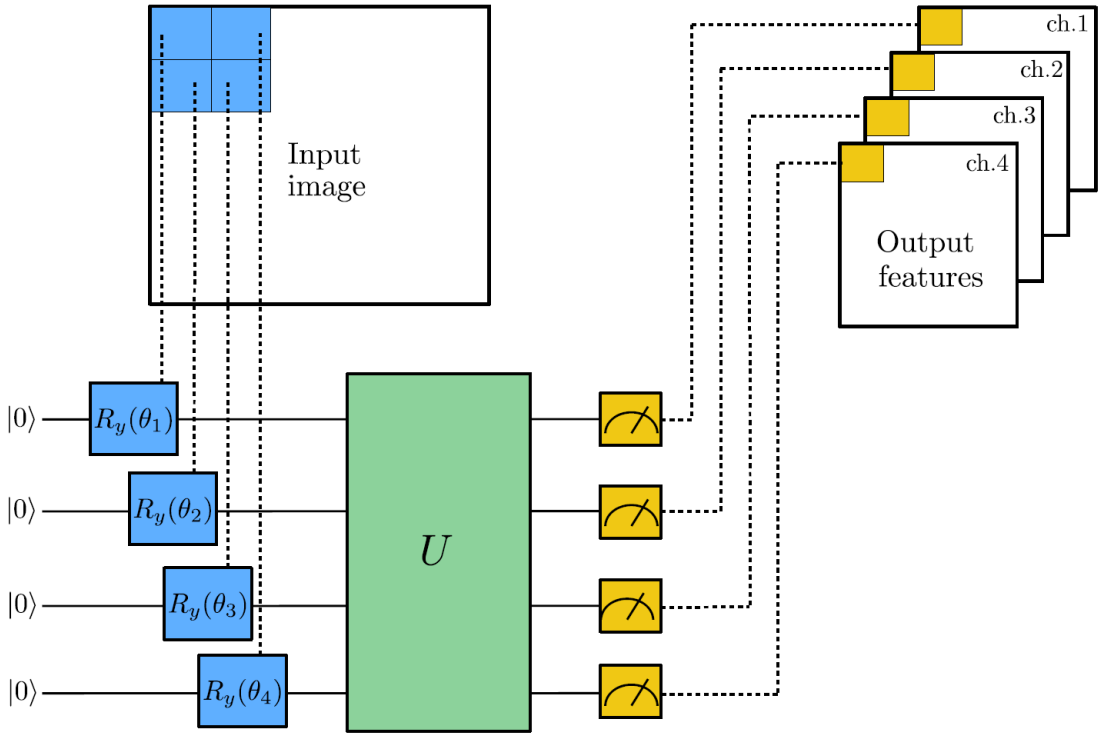
```
Requirement already satisfied: tensorflow in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (2.21.0)
Requirement already satisfied: absl-py>=1.0.0 in /home/msoliviawatt/machine_
learning/ml-env/lib/python3.13/site-packages (from tensorflow) (2.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /home/msoliviawatt/machi
ne_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=25.9.23 in /home/msoliviawatt/ma
chine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (25.12.
19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /home/
msoliviawatt/machine_learning/ml-env/lib/python3.13/site-packages (from tens
orflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in /home/msoliviawatt/mac
hine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /home/msoliviawatt/machin
e_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /home/msoliviawatt/machi
ne_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (from tensorflow) (26.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (7.
34.1)
Requirement already satisfied: requests<3,>=2.21.0 in /home/msoliviawatt/mac
hine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (2.32.5)
Requirement already satisfied: setuptools in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (from tensorflow) (81.0.0)
Requirement already satisfied: six>=1.12.0 in /home/msoliviawatt/machine_lea
rning/ml-env/lib/python3.13/site-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /home/msoliviawatt/machin
e_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (4.
15.0)
Requirement already satisfied: wrapt>=1.11.0 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /home/msoliviawatt/mac
hine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (1.80.0)
Requirement already satisfied: keras>=3.12.0 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from tensorflow) (3.14.0)
Requirement already satisfied: numpy>=1.26.0 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from tensorflow) (2.4.2)
Requirement already satisfied: h5py<3.15.0,>=3.11.0 in /home/msoliviawatt/ma
chine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (3.14.
0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from tensorflow) (0.
5.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from requests<3,>=2.
21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /home/msoliviawatt/machine_le
arning/ml-env/lib/python3.13/site-packages (from requests<3,>=2.21.0->tensor
flow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from requests<3,>=2.21.0->
```

```
tensorflow) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from requests<3,>=2.21.0->
tensorflow) (2026.1.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from astunparse>=1.6.0->te
nsorflow) (0.46.3)
Requirement already satisfied: rich in /home/msoliviawatt/machine_learning/m
l-env/lib/python3.13/site-packages (from keras>=3.12.0->tensorflow) (15.0.0)
Requirement already satisfied: namex in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (from keras>=3.12.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from keras>=3.12.0->tensorflow) (0.1
9.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /home/msoliviawatt/m
achine_learning/ml-env/lib/python3.13/site-packages (from rich->keras>=3.12.
0->tensorflow) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from rich->keras>=3.
12.0->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (from markdown-it-py>=2.2.0->rich->
keras>=3.12.0->tensorflow) (0.1.2)
Note: you may need to restart the kernel to use updated packages.
```

Quanvolutional Neural Networks {#quanvolution}

> > Warning
>
> This demo uses TensorFlow, which is a deprecated interface with PennyLane v0.42. Interfacing with TensorFlow will no longer be supported with PennyLane v0.43 and higher. Consider switching to a different machine learning interface with PennyLane, like PyTorch or JAX.

In this demo we implement the *Quanvolutional Neural Network*, a quantum machine learning model originally introduced in Henderson et al. (2019).



Introduction

Classical convolution

The *convolutional neural network* (CNN) is a standard model in classical machine learning which is particularly suitable for processing images. The model is based on the idea of a *convolution layer* where, instead of processing the full input data with a global function, a local convolution is applied.

If the input is an image, small local regions are sequentially processed with the same kernel. The results obtained for each region are usually associated to different channels of a single output pixel. The union of all the output pixels produces a new image-like object, which can be further processed by additional layers.

Quantum convolution

One can extend the same idea also to the context of quantum variational circuits. A possible approach is given by the following procedure which is very similar to the one used in Ref. [1]. The scheme is also represented in the figure at the top of this tutorial.

1. A small region of the input image, in our example a 2×2 square, is embedded into a quantum circuit. In this demo, this is achieved with

parametrized rotations applied to the qubits initialized in the ground state.
2. A quantum computation, associated to a unitary U, is performed on the system. The unitary could be generated by a variational quantum circuit or, more simply, by a random circuit as proposed in Ref. [1].
3. The quantum system is finally measured, obtaining a list of classical expectation values. The measurement results could also be classically post-processed as proposed in Ref. [1] but, for simplicity, in this demo we directly use the raw expectation values.
4. Analogously to a classical convolution layer, each expectation value is mapped to a different channel of a single output pixel.
5. Iterating the same procedure over different regions, one can scan the full input image, producing an output object which will be structured as a multi-channel image.
6. The quantum convolution can be followed by further quantum layers or by classical layers.

The main difference with respect to a classical convolution is that a quantum circuit can generate highly complex kernels whose computation could be, at least in principle, classically intractable.

> > Note
>
> In this tutorial we follow the approach of Ref. [1] in which a fixed non-trainable quantum circuit is used as a "quanvolution" kernel, while the subsequent classical layers are trained for the classification problem of interest. However, by leveraging the ability of PennyLane to evaluate gradients of quantum circuits, the quantum kernel could also be trained.

General setup

This Python code requires *PennyLane* with the *TensorFlow* interface and the plotting library *matplotlib*.

In [25]:
```
import pennylane as qml
from pennylane import numpy as np
from pennylane.templates import RandomLayers
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

import os
```

Setting of the main hyper-parameters of the model

In [26]:
```
n_epochs = 30   # Number of optimization epochs
n_layers = 1    # Number of random layers
n_train = 50    # Size of the train dataset
n_test = 30     # Size of the test dataset

# SAVE_PATH = "../_static/demonstration_assets/quanvolution/"  # Data saving
SAVE_PATH = "./quanv_data/"
os.makedirs(SAVE_PATH, exist_ok=True)
PREPROCESS = True           # If False, skip quantum processing and load dat
np.random.seed(0)           # Seed for NumPy random number generator
tf.random.set_seed(0)       # Seed for TensorFlow random number generator
```

Loading of the Stanford Cars dataset

In [27]:
```
from torchvision.datasets import StanfordCars
from torchvision import transforms
from torch.utils.data import Dataset, Subset, random_split, DataLoader
import torch

base_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((28, 28)),
    transforms.ToTensor()
])

full_data = StanfordCars(
    root="./data",
    split="train",
    download=False,
    transform=base_transform
)

print(len(full_data.classes))
```

```
196
```

In [28]:
```
for i, name in enumerate(full_data.classes[:10]):
    print(i, name)
```

```
0 AM General Hummer SUV 2000
1 Acura RL Sedan 2012
2 Acura TL Sedan 2012
3 Acura TL Type-S 2008
4 Acura TSX Sedan 2012
5 Acura Integra Type R 2001
6 Acura ZDX Hatchback 2012
7 Aston Martin V8 Vantage Convertible 2012
8 Aston Martin V8 Vantage Coupe 2012
9 Aston Martin Virage Convertible 2012
```

In [29]:
```
selected_classes = [0, 1, 2, 3]
```

In [30]:
```
# manually splitting the data
selected_indices = [
    i for i, (_, label) in enumerate(full_data._samples)
    if label in selected_classes
]

filtered_data = Subset(full_data, selected_indices)

class_map = {old_label: new_label for new_label, old_label in enumerate(sele

class RemappedSubset(Dataset):
    def __init__(self, subset, class_map):
        self.subset = subset
        self.class_map = class_map

    def __len__(self):
        return len(self.subset)

    def __getitem__(self, idx):
        image, label = self.subset[idx]
        return image, self.class_map[label]

remapped_data = RemappedSubset(filtered_data, class_map)

total_size = len(remapped_data)
train_size = int(0.8 * total_size)
val_size = total_size - train_size

generator = torch.Generator().manual_seed(42)

train_subset, val_subset = random_split(
    remapped_data,
    [train_size, val_size],
    generator=generator
)

train_dl = DataLoader(train_subset, batch_size=16, shuffle=True)
val_dl = DataLoader(val_subset, batch_size=16, shuffle=False)

print("Total filtered samples:", len(remapped_data))
print("Train samples:", len(train_subset))
```

```
print("Val samples:", len(val_subset))
print("Class map:", class_map)
```

```
Total filtered samples: 162
Train samples: 129
Val samples: 33
Class map: {0: 0, 1: 1, 2: 2, 3: 3}
```

Quantum circuit as a convolution kernel

We follow the scheme described in the introduction and represented in the figure at the top of this demo.

We initialize a PennyLane `default.qubit` device, simulating a system of 4 qubits. The associated `qnode` represents the quantum circuit consisting of:

1. an embedding layer of local R_y rotations (with angles scaled by a factor of π);
2. a random circuit of `n_layers` ;
3. a final measurement in the computational basis, estimating 4 expectation values.

In [31]:
```
dev = qml.device("default.qubit", wires=4)
# Random circuit parameters
rand_params = np.random.uniform(high=2 * np.pi, size=(n_layers, 4))

@qml.qnode(dev)
def circuit(phi):
    # Encoding of 4 classical input values
    for j in range(4):
        qml.RY(np.pi * phi[j], wires=j)

    # Random quantum circuit
    RandomLayers(rand_params, wires=list(range(4)))

    # Measurement producing 4 classical output values
    return [qml.expval(qml.PauliZ(j)) for j in range(4)]
```

The next function defines the convolution scheme:

1. the image is divided into squares of 2×2 pixels;
2. each square is processed by the quantum circuit;
3. the 4 expectation values are mapped into 4 different channels of a single output pixel.

> Note

> This process halves the resolution of the input image. In the standard language of CNN, this would correspond to a convolution with a 2×2 *kernel* and a *stride* equal to 2.

In [32]:
```
def quanv(image):
    """Convolves the input image with many applications of the same quantum
    out = np.zeros((14, 14, 4))

    # Loop over the coordinates of the top-left pixel of 2X2 squares
    for j in range(0, 28, 2):
        for k in range(0, 28, 2):
            # Process a squared 2x2 region of the image with a quantum circu
            q_results = circuit(
                [
                    image[j, k, 0],
                    image[j, k + 1, 0],
                    image[j + 1, k, 0],
                    image[j + 1, k + 1, 0]
                ]
            )
            # Assign expectation values to different channels of the output
            for c in range(4):
                out[j // 2, k // 2, c] = q_results[c]
    return out
```

In [33]:
```
n_train = 100
n_test = 40

train_subset_small = Subset(train_subset, range(min(n_train, len(train_subse
val_subset_small = Subset(val_subset, range(min(n_test, len(val_subset))))

def dataset_to_numpy(dataset):
    images = []
    labels = []

    for img, label in dataset:
        # img is [C, H, W], but PennyLane tutorial expects [H, W, C]
        img_np = img.permute(1, 2, 0).numpy()
        images.append(img_np)
        labels.append(label)

    images = np.array(images)
    labels = np.array(labels)

    return images, labels

train_images, train_labels = dataset_to_numpy(train_subset_small)
test_images, test_labels = dataset_to_numpy(val_subset_small)

print("train_images shape: ", train_images.shape)
print("train_labels shape: ", train_labels.shape)
print("test_images shape: ", test_images.shape)
print("test_labels shape: ", test_labels.shape)
```

```
print("Unique train labels: ", np.unique(train_labels))
print("Unique test labels: ", np.unique(test_labels))
```

```
train_images shape:  (100, 28, 28, 1)
train_labels shape:  (100,)
test_images shape:  (33, 28, 28, 1)
test_labels shape:  (33,)
Unique train labels:  [0 1 2 3]
Unique test labels:  [0 1 2 3]
```

Quantum pre-processing of the dataset

Since we are not going to train the quantum convolution layer, it is more efficient to apply it as a "pre-processing" layer to all the images of our dataset. Later an entirely classical model will be directly trained and tested on the pre-processed dataset, avoiding unnecessary repetitions of quantum computations.

The pre-processed images will be saved in the folder `SAVE_PATH`. Once saved, they can be directly loaded by setting `PREPROCESS = False`, otherwise the quantum convolution is evaluated at each run of the code.

In [34]:
```
if PREPROCESS == True:
    q_train_images = []
    print("Quantum pre-processing of train images:")
    for idx, img in enumerate(train_images):
        print("{}/{}        ".format(idx + 1, n_train), end="\r")
        q_train_images.append(quanv(img))
    q_train_images = np.asarray(q_train_images)

    q_test_images = []
    print("\nQuantum pre-processing of test images:")
    for idx, img in enumerate(test_images):
        print("{}/{}        ".format(idx + 1, n_test), end="\r")
        q_test_images.append(quanv(img))
    q_test_images = np.asarray(q_test_images)

    # Save pre-processed images
    np.save(SAVE_PATH + "q_train_images.npy", q_train_images)
    np.save(SAVE_PATH + "q_test_images.npy", q_test_images)


# Load pre-processed images
q_train_images = np.load(SAVE_PATH + "q_train_images.npy")
q_test_images = np.load(SAVE_PATH + "q_test_images.npy")
```

```
Quantum pre-processing of train images:
100/100
Quantum pre-processing of test images:
33/40
```

Let us visualize the effect of the quantum convolution layer on a batch of samples:

In [35]:

```
n_samples = 4
n_channels = 4
fig, axes = plt.subplots(1 + n_channels, n_samples, figsize=(10, 10))
for k in range(n_samples):
    axes[0, 0].set_ylabel("Input")
    if k != 0:
        axes[0, k].yaxis.set_visible(False)
    axes[0, k].imshow(train_images[k, :, :, 0], cmap="gray")

    # Plot all output channels
    for c in range(n_channels):
        axes[c + 1, 0].set_ylabel("Output [ch. {}]".format(c))
        if k != 0:
            axes[c, k].yaxis.set_visible(False)
        axes[c + 1, k].imshow(q_train_images[k, :, :, c], cmap="gray")

plt.tight_layout()
plt.show()
```

Below each input image, the 4 output channels generated by the quantum convolution are visualized in gray scale.

One can clearly notice the downsampling of the resolution and some local distortion introduced by the quantum kernel. On the other hand the global shape of the image is preserved, as expected for a convolution layer.

Hybrid quantum-classical model

After the application of the quantum convolution layer we feed the resulting features into a classical neural network that will be trained to classify the 10 different digits of the MNIST dataset.

We use a very simple model: just a fully connected layer with 10 output nodes with a final *softmax* activation function.

The model is compiled with a *stochastic-gradient-descent* optimizer, and a *cross-entropy* loss function.

In [36]:
```
def MyModel():
    """Initializes and returns a custom Keras model
    which is ready to be trained."""
    model = keras.models.Sequential([
        keras.layers.Flatten(),
        # keras.layers.Dense(10, activation="softmax")
        keras.layers.Dense(4, activation="softmax")
    ])

    model.compile(
        optimizer='adam',
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

Training

We first initialize an instance of the model, then we train and validate it with the dataset that has been already pre-processed by a quantum convolution.

In [37]:
```
q_model = MyModel()

q_history = q_model.fit(
    q_train_images,
    train_labels,
    validation_data=(q_test_images, test_labels),
    batch_size=4,
    epochs=n_epochs,
    verbose=2,
)
```

```
Epoch 1/30
E0000 00:00:1776249942.546873    9309 cuda_executor.cc:1737] INTERNAL: CUDA
Runtime error: Failed call to cudaGetRuntimeVersion: Error loading CUDA libr
aries. GPU will not be used.: Error loading CUDA libraries. GPU will not be
used.
W0000 00:00:1776249942.594988    9309 gpu_device.cc:2365] Cannot dlopen some
GPU libraries. Please make sure the missing libraries mentioned above are in
stalled properly if you would like to use GPU. Follow the guide at https://w
ww.tensorflow.org/install/gpu for how to download and setup the required lib
raries for your platform.
Skipping registering GPU devices...
```

```
25/25 - 0s - 20ms/step - accuracy: 0.3000 - loss: 1.5090 - val_accuracy: 0.2
121 - val_loss: 1.4979
Epoch 2/30
25/25 - 0s - 3ms/step - accuracy: 0.4100 - loss: 1.2378 - val_accuracy: 0.33
33 - val_loss: 1.3892
Epoch 3/30
25/25 - 0s - 3ms/step - accuracy: 0.5500 - loss: 1.0934 - val_accuracy: 0.33
33 - val_loss: 1.3840
Epoch 4/30
25/25 - 0s - 3ms/step - accuracy: 0.6200 - loss: 0.9965 - val_accuracy: 0.39
39 - val_loss: 1.3845
Epoch 5/30
25/25 - 0s - 3ms/step - accuracy: 0.6800 - loss: 0.9167 - val_accuracy: 0.36
36 - val_loss: 1.3894
Epoch 6/30
25/25 - 0s - 3ms/step - accuracy: 0.7000 - loss: 0.8483 - val_accuracy: 0.39
39 - val_loss: 1.3944
Epoch 7/30
25/25 - 0s - 3ms/step - accuracy: 0.7300 - loss: 0.7883 - val_accuracy: 0.39
39 - val_loss: 1.3993
Epoch 8/30
25/25 - 0s - 3ms/step - accuracy: 0.7900 - loss: 0.7347 - val_accuracy: 0.42
42 - val_loss: 1.4041
Epoch 9/30
25/25 - 0s - 3ms/step - accuracy: 0.8300 - loss: 0.6863 - val_accuracy: 0.42
42 - val_loss: 1.4090
Epoch 10/30
25/25 - 0s - 3ms/step - accuracy: 0.8400 - loss: 0.6423 - val_accuracy: 0.42
42 - val_loss: 1.4142
Epoch 11/30
25/25 - 0s - 3ms/step - accuracy: 0.8700 - loss: 0.6021 - val_accuracy: 0.42
42 - val_loss: 1.4196
Epoch 12/30
25/25 - 0s - 3ms/step - accuracy: 0.8900 - loss: 0.5652 - val_accuracy: 0.42
42 - val_loss: 1.4254
Epoch 13/30
25/25 - 0s - 3ms/step - accuracy: 0.9200 - loss: 0.5313 - val_accuracy: 0.42
42 - val_loss: 1.4315
Epoch 14/30
25/25 - 0s - 3ms/step - accuracy: 0.9400 - loss: 0.5001 - val_accuracy: 0.45
45 - val_loss: 1.4380
Epoch 15/30
25/25 - 0s - 3ms/step - accuracy: 0.9500 - loss: 0.4713 - val_accuracy: 0.42
42 - val_loss: 1.4449
Epoch 16/30
25/25 - 0s - 3ms/step - accuracy: 0.9600 - loss: 0.4447 - val_accuracy: 0.42
42 - val_loss: 1.4520
Epoch 17/30
25/25 - 0s - 3ms/step - accuracy: 0.9900 - loss: 0.4201 - val_accuracy: 0.42
42 - val_loss: 1.4594
Epoch 18/30
25/25 - 0s - 3ms/step - accuracy: 0.9900 - loss: 0.3972 - val_accuracy: 0.42
42 - val_loss: 1.4670
Epoch 19/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.3761 - val_accuracy: 0.42
42 - val_loss: 1.4748
```

```
Epoch 20/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.3564 - val_accuracy: 0.42
42 - val_loss: 1.4828
Epoch 21/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.3381 - val_accuracy: 0.42
42 - val_loss: 1.4908
Epoch 22/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.3210 - val_accuracy: 0.42
42 - val_loss: 1.4990
Epoch 23/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.3052 - val_accuracy: 0.42
42 - val_loss: 1.5073
Epoch 24/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2904 - val_accuracy: 0.42
42 - val_loss: 1.5156
Epoch 25/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2765 - val_accuracy: 0.42
42 - val_loss: 1.5239
Epoch 26/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2636 - val_accuracy: 0.42
42 - val_loss: 1.5322
Epoch 27/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2515 - val_accuracy: 0.42
42 - val_loss: 1.5406
Epoch 28/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2401 - val_accuracy: 0.42
42 - val_loss: 1.5489
Epoch 29/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2295 - val_accuracy: 0.42
42 - val_loss: 1.5573
Epoch 30/30
25/25 - 0s - 3ms/step - accuracy: 1.0000 - loss: 0.2195 - val_accuracy: 0.39
39 - val_loss: 1.5656
```

In order to compare the results achievable with and without the quantum convolution layer, we initialize also a "classical" instance of the model that will be directly trained and validated with the raw MNIST images (i.e., without quantum pre-processing).

In [38]:
```
c_model = MyModel()

c_history = c_model.fit(
    train_images,
    train_labels,
    validation_data=(test_images, test_labels),
    batch_size=4,
    epochs=n_epochs,
    verbose=2,
)
```

```
Epoch 1/30
25/25 - 1s - 22ms/step - accuracy: 0.2400 - loss: 1.5499 - val_accuracy: 0.2
121 - val_loss: 1.4901
Epoch 2/30
25/25 - 0s - 3ms/step - accuracy: 0.2700 - loss: 1.4113 - val_accuracy: 0.21
21 - val_loss: 1.5000
Epoch 3/30
25/25 - 0s - 3ms/step - accuracy: 0.3500 - loss: 1.3550 - val_accuracy: 0.27
27 - val_loss: 1.4826
Epoch 4/30
25/25 - 0s - 3ms/step - accuracy: 0.4100 - loss: 1.3035 - val_accuracy: 0.27
27 - val_loss: 1.5066
Epoch 5/30
25/25 - 0s - 3ms/step - accuracy: 0.4700 - loss: 1.2687 - val_accuracy: 0.21
21 - val_loss: 1.5245
Epoch 6/30
25/25 - 0s - 3ms/step - accuracy: 0.5000 - loss: 1.2384 - val_accuracy: 0.18
18 - val_loss: 1.5384
Epoch 7/30
25/25 - 0s - 3ms/step - accuracy: 0.5000 - loss: 1.2109 - val_accuracy: 0.21
21 - val_loss: 1.5510
Epoch 8/30
25/25 - 0s - 3ms/step - accuracy: 0.5300 - loss: 1.1856 - val_accuracy: 0.21
21 - val_loss: 1.5620
Epoch 9/30
25/25 - 0s - 3ms/step - accuracy: 0.5900 - loss: 1.1620 - val_accuracy: 0.21
21 - val_loss: 1.5717
Epoch 10/30
25/25 - 0s - 3ms/step - accuracy: 0.5900 - loss: 1.1398 - val_accuracy: 0.18
18 - val_loss: 1.5805
Epoch 11/30
25/25 - 0s - 3ms/step - accuracy: 0.6000 - loss: 1.1189 - val_accuracy: 0.18
18 - val_loss: 1.5885
Epoch 12/30
25/25 - 0s - 3ms/step - accuracy: 0.6000 - loss: 1.0990 - val_accuracy: 0.18
18 - val_loss: 1.5959
Epoch 13/30
25/25 - 0s - 3ms/step - accuracy: 0.6000 - loss: 1.0800 - val_accuracy: 0.21
21 - val_loss: 1.6029
Epoch 14/30
25/25 - 0s - 3ms/step - accuracy: 0.6100 - loss: 1.0619 - val_accuracy: 0.21
21 - val_loss: 1.6096
Epoch 15/30
25/25 - 0s - 3ms/step - accuracy: 0.6200 - loss: 1.0445 - val_accuracy: 0.21
21 - val_loss: 1.6161
Epoch 16/30
25/25 - 0s - 3ms/step - accuracy: 0.6300 - loss: 1.0278 - val_accuracy: 0.21
21 - val_loss: 1.6223
Epoch 17/30
25/25 - 0s - 3ms/step - accuracy: 0.6300 - loss: 1.0117 - val_accuracy: 0.21
21 - val_loss: 1.6284
Epoch 18/30
25/25 - 0s - 3ms/step - accuracy: 0.6500 - loss: 0.9962 - val_accuracy: 0.21
21 - val_loss: 1.6344
Epoch 19/30
25/25 - 0s - 3ms/step - accuracy: 0.6700 - loss: 0.9812 - val_accuracy: 0.21
```

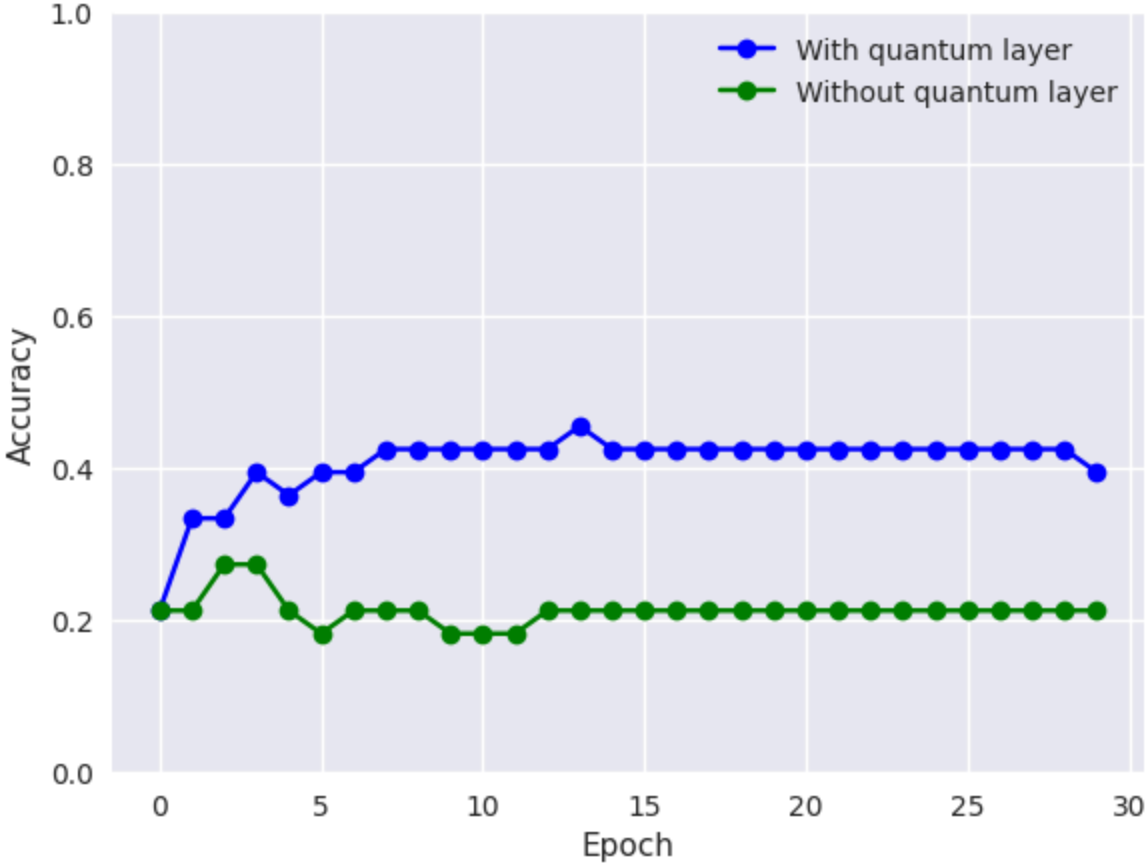
```
21 - val_loss: 1.6403
Epoch 20/30
25/25 - 0s - 3ms/step - accuracy: 0.6700 - loss: 0.9667 - val_accuracy: 0.21
21 - val_loss: 1.6462
Epoch 21/30
25/25 - 0s - 3ms/step - accuracy: 0.6900 - loss: 0.9527 - val_accuracy: 0.21
21 - val_loss: 1.6519
Epoch 22/30
25/25 - 0s - 3ms/step - accuracy: 0.7000 - loss: 0.9391 - val_accuracy: 0.21
21 - val_loss: 1.6576
Epoch 23/30
25/25 - 0s - 3ms/step - accuracy: 0.7000 - loss: 0.9259 - val_accuracy: 0.21
21 - val_loss: 1.6633
Epoch 24/30
25/25 - 0s - 3ms/step - accuracy: 0.7100 - loss: 0.9131 - val_accuracy: 0.21
21 - val_loss: 1.6689
Epoch 25/30
25/25 - 0s - 3ms/step - accuracy: 0.7300 - loss: 0.9006 - val_accuracy: 0.21
21 - val_loss: 1.6745
Epoch 26/30
25/25 - 0s - 3ms/step - accuracy: 0.7500 - loss: 0.8884 - val_accuracy: 0.21
21 - val_loss: 1.6801
Epoch 27/30
25/25 - 0s - 3ms/step - accuracy: 0.7600 - loss: 0.8766 - val_accuracy: 0.21
21 - val_loss: 1.6856
Epoch 28/30
25/25 - 0s - 3ms/step - accuracy: 0.7600 - loss: 0.8651 - val_accuracy: 0.21
21 - val_loss: 1.6911
Epoch 29/30
25/25 - 0s - 3ms/step - accuracy: 0.7600 - loss: 0.8538 - val_accuracy: 0.21
21 - val_loss: 1.6965
Epoch 30/30
25/25 - 0s - 3ms/step - accuracy: 0.7700 - loss: 0.8428 - val_accuracy: 0.21
21 - val_loss: 1.7020
```

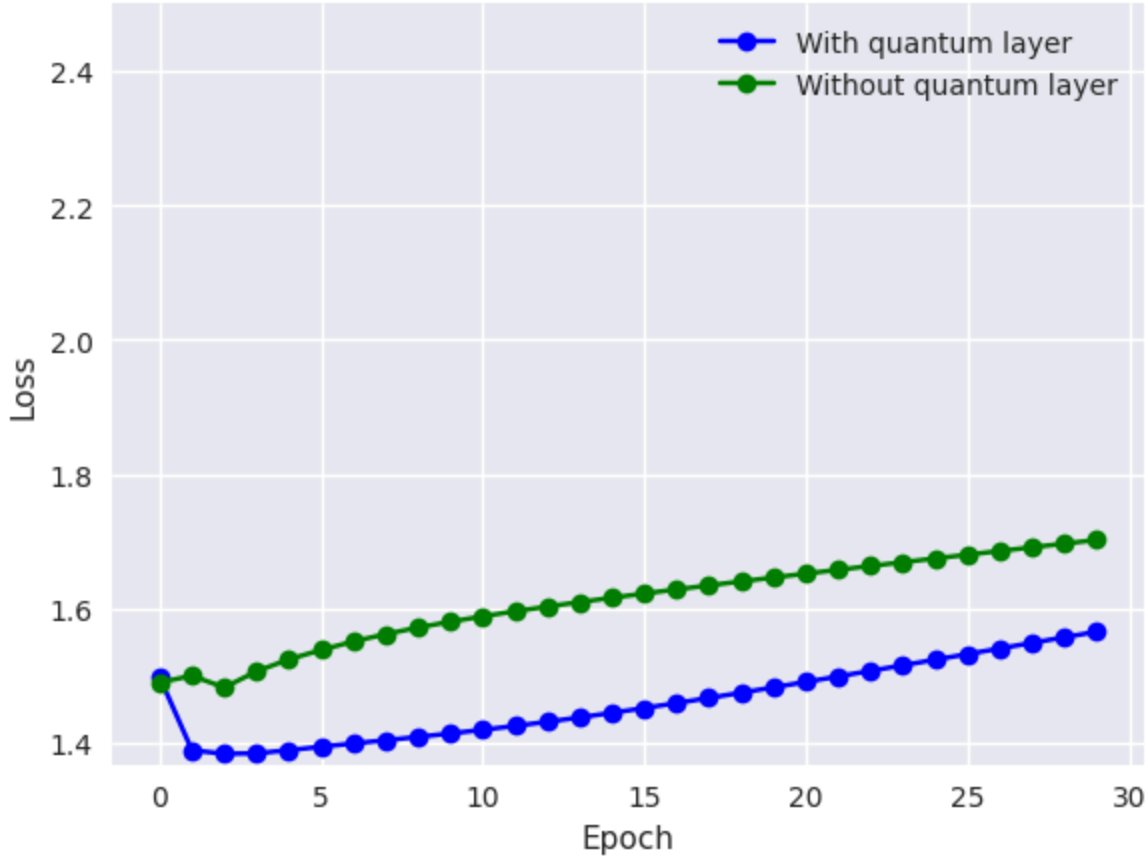
Results

We can finally plot the test accuracy and the test loss with respect to the number of training epochs.

In [39]:
```
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8")
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6, 9))

ax1.plot(q_history.history["val_accuracy"], "-ob", label="With quantum layer
ax1.plot(c_history.history["val_accuracy"], "-og", label="Without quantum la
ax1.set_ylabel("Accuracy")
ax1.set_ylim([0, 1])
ax1.set_xlabel("Epoch")
ax1.legend()

ax2.plot(q_history.history["val_loss"], "-ob", label="With quantum layer")
```

```
ax2.plot(c_history.history["val_loss"], "-og", label="Without quantum layer"
ax2.set_ylabel("Loss")
ax2.set_ylim(top=2.5)
ax2.set_xlabel("Epoch")
ax2.legend()
plt.tight_layout()
plt.show()
```


With quantum layer
Without quantum layer
Accuracy
Epoch
1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30


With quantum layer
Without quantum layer
Loss
Epoch
2.4
2.2
2.0
1.8
1.6
1.4
0
5
10
15
20
25
30

1. Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, Tristan Cook. "Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits." arXiv:1904.04767, 2019.